

CCF-GESP C++四级

编程能力等级认证



第十课

二维数组 4



真题解析

(2023年6月) 图像压缩

【问题描述】

图像是由很多的像素点组成的。如果用0表示黑，255表示白，0和255之间的值代表不同程度的灰色，则可以用一个字节表达一个像素（取值范围为十进制0-255、十六进制00-FF）。这样的像素组成的图像，称为**256级灰阶**的灰度图像。

0

255



现在希望将256级灰阶的灰度图像压缩为16级灰阶，即每个像素的取值范围为十进制0-15、十六进制0-F。**压缩规则为：**

- ① 统计出每种灰阶的数量，取数量**最多的前16种灰阶**（如某种灰阶的数量与另外一种灰阶的数量相同，则以灰阶值**从小到大**为序），分别编号**0-F**（最多的编号为0，以此类推）。
- ② 其他灰阶转换到**最近的**16种灰阶之一，将某个点灰阶数与16种灰阶中的一种相减，绝对值**最小**即为最近，如果绝对值相等，则编号较小的灰阶更近。



真题解析

(2023年6月) 图像压缩

【输入格式】

输入第 1 行为一个正整数 N ，表示接下来有 N 行数据组成一副256级灰阶的灰度图像。约定 $10 \leq N \leq 20$ 。

第2行开始的行，每行为长度相等且为**偶数的字符串**，每**两个字符**用十六进制表示一个像素。约定输入的灰度图像至少有16种灰阶。约定每行最多20个像素。

【输出格式】

第一行输出压缩选定的 16 种灰阶的十六进制编码，共计 32 个字符。

第二行开始的行，输出压缩后的图像，每个像素一位十六进制数表示压缩后的灰阶值。

一个像素的灰阶值

00FFCFAB00FFAC09071B5CCFAB76

00 FF CF AB 00 FF AC 09 07 1B 5C CF AB 76

14个像素



真题解析

(2023年6月) 图像压缩

【输入样例】

10

00FFCFAB00FFAC09071B5CCFAB76
00AFCBAB11FFAB09981D34CFAF56
01BFCEAB00FFAC0907F25FCFBA65
10FBCBAB11FFAB09981DF4CFCA67
00FFCBFB00FFAC0907A25CCFFC76
00FFCBAB1CFFCB09FC1AC4CFCF67
01FCCBAB00FFAC0F071A54CFBA65
10EFCBAB11FFAB09981B34CFCF67
01FFCBAB00FFAC0F071054CFAC76
1000CBAB11FFAB0A981B84CFCF66

【输出样例】

ABCFFF00CB09AC07101198011B6776FC

最多的前16种灰阶

321032657CD10E
36409205ACC16D
B41032657FD16D
8F409205ACF14D
324F326570D1FE
3240C245FC411D
BF4032687CD16D
8F409205ACC11D
B240326878D16E
83409205ACE11D

【样例解释 1】

灰阶'AB'、'CF'和'FF'出现 14 次，'00'出现 10 次，'CB'出现9 次，'09'出现 7 次，'AC'出现 6 次，'07'出现 5 次，'10'、'11'和'98'出现 4 次，'01'、'1B'、'67'、'76'和'FC'出现 3 次。



真题解析

图像压缩规则分析

16种灰阶值

编号

AB	CF	FF	00	CB	09	AC	07	10	11	98	01	1B	67	76	FC
0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F

00FFCFAB00FFAC09071B5CCFAB76
00AFCBAB11FFAB09981D34CFAF56
01BFCEAB00FFAC0907F25FCFBA65
10FBCBAB11FFAB09981DF4CFCA67
00FFCBFB00FFAC0907A25CCFFC76
00FFCBAB1CFFCB09FC1AC4CFCF67
01FCCBAB00FFAC0F071A54CFBA65
10EFCBAB11FFAB09981B34CFCF67
01FFCBAB00FFAC0F071054CFAC76
1000CBAB11FFAB0A981B84CFCF66

原图

压缩

3 2

转换为最接近灰阶的编号

灰阶值由**两位**十六进制数
压缩成了**一位**十六进制数



真题解析

图像压缩规则分析

16种灰阶值

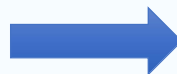
编号

AB	CF	FF	00	CB	09	AC	07	10	11	98	01	1B	67	76	FC
0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F

00FFCFAB00FFAC09071B5CCFAB76
00AFCBAB11FFAB09981D34CFAF56
01BFCEAB00FFAC0907F25FCFBA65
10FBCBAB11FFAB09981DF4CFCA67
00FFCBFB00FFAC0907A25CCFFC76
00FFCBAB1CFFCB09FC1AC4CFCF67
01FCCBAB00FFAC0F071A54CFBA65
10EFCBAB11FFAB09981B34CFCF67
01FFCBAB00FFAC0F071054CFAC76
1000CBAB11FFAB0A981B84CFCF66

原图

压缩



321032657CD10E
36409205ACC16D
B41032657FD16D
8F409205ACF14D
324F326570D1FE
3240C245FC411D
BF4032687CD16D
8F409205ACC11D
B240326878D16E
83409205ACE11D

宽度为原图的一半



真题解析

思路分析

把图像字符串
转为**数值**



找出**最多的**
16种灰阶



根据规则
压缩图像



输出压缩后
图像的灰阶值



真题解析

实现步骤

- ① 根据题目要求**输入** n ($10 \leq N \leq 20$)，以及 **n 行**图像的灰阶值的**字符串**；
- ② 将图像灰阶字符**转换成数值**，保存在二维数组**image**中；

3F → 63

```
// 将单个字符转换为一位十六进制数
int trans(char a){
    .....
}
```

```
int image[25][25];
int main(){
    int n;
    cin >> n;
    char line[50]={0};
    for(int i=0;i<n;i++){
        cin >> line;
        for(int j=0;j<strlen(line);j+=2){
            int c = 将字符串灰阶值转换为十进制数;
            image[i][j/2]=c;
        }
    }
    return 0;
}
```

每行最多20个像素，
字符串的长度最多40

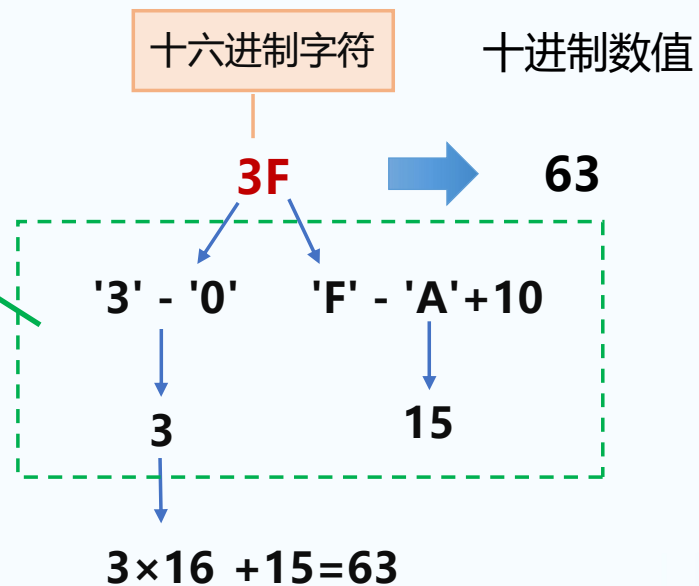
两个字符为一个像素，
循环步长为2



真题解析

trans函数的实现

```
// 将单个字符转换为一位十六进制数  
int trans(char a){  
    if (a <= '9')  
        return (a - '0');  
    return (a - 'A' + 10);  
}
```





真题解析

实现步骤

- ① 根据题目要求**输入** n ($10 \leq N \leq 20$)，以及 **n 行**图像的灰阶值的**字符串**；
- ② 将图像灰阶字符**转换成数值**，保存在二维数组**image**中；

```
// 将单个字符转换为一位十六进制数
int trans(char a){
    if (a <= '9')
        return (a - '0');
    return (a - 'A' + 10);
}
```

```
int image[25][25];
int main(){
    int n;
    cin >> n;
    char line[50] = {0};
    for(int i = 0; i < n; i++){
        cin >> line;
        for(int j = 0; j < strlen(line); j += 2){
            int c = trans(line[j]) * 16 + trans(line[j + 1]);
            image[i][j / 2] = c;
        }
    }
    return 0;
}
```



真题解析

思路分析

把图像字符串
转为**数值**



找出**最多的**
16种灰阶



根据规则
压缩图像



输出压缩后
图像的灰阶值



真题解析

实现步骤

③ 找出出现次数最多的**16种**灰阶值，并进行**0~F**编号；

- 使用**桶排序的思想**统计每个灰阶值出现的次数

```
int his[256]; // 用于统计灰阶值
int color[16]; // 用于保存最多的16种灰阶值
int main(){
    .....
    for(int i=0;i<n;i++){
        cin>>line;
        for(int j=0;j<strlen(line);j+=2){
            int c = trans(line[j])*16+trans(line[j+1]);
            image[i][j/2]=c;
            his[c]++; // 对灰阶值进行计数
        }
    }
    .....
    return 0;
}
```



真题解析

实现步骤

- ③ 找出出现次数最多的**16种**灰阶值，并进行**0~F**编号；
- 使用**桶排序的思想**统计每个灰阶值出现的次数
 - 找出最多的前**16种**灰阶值，保存在数组**color**，**数组下标**即为编号

```
for(int i=0;i<16;i++){  
    int max=0; // 记录最大值下标  
    for(int j=0;j<256;j++){  
        if(his[max]<his[j])max=j;  
    }  
    color[i]=max;  
    his[max]=-1; // 标记为-1, 该灰阶值已获取  
}
```

在his数组中找16次最大值的下标

数组color

下标

AB	CF	FF	00	CB	09	AC	07	10	11	98	01	1B	67	76	FC
0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F



真题解析

思路分析

把图像字符串
转为**数值**



找出**最多的**
16种灰阶



根据规则
压缩图像



输出压缩后
图像的灰阶值



真题解析

实现步骤

④ 对数组image中的元素进行压缩处理

- 遍历image数组，将压缩之后的灰阶值保存在一个新数组cpimg中

CF → 1

```
// 计算整数c的压缩值
int compress(int c){
    .....
}
```

```
int cpimg[25][25]; // 用于保存压缩图
int main(){
    .....
    int m=strlen(line)/2;
    for(int i=0;i<n;i++){
        for(int j=0;j<m;j++){
            cpimg[i][j] = // 计算image[i][j]的压缩值;
        }
    }
    return 0;
}
```

计算数组image
的列长度



真题解析

compress函数的实现

```
// 计算整数c的压缩值
int compress(int c){
    int min=255,min_i=0;
    for(int h=0;h<16;h++){
        int k=abs(c-color[h]);
        if(k<min){
            min=k;
            min_i=h;}
    }
    return min_i;
}
```

压缩规则:

- 在color数组中找到与当前元素最接近的灰阶值;
- 与color数组中其中一个灰阶值相减, 绝对值最小即为最接近;
- 最接近的灰阶值所对应的下标即为压缩值。

数组color

下标

AB	CF	FF	00	CB	09	AC	07	10	11	98	01	1B	67	76	FC
0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F



真题解析

实现步骤

④ 对数组image中的元素进行压缩处理

- 遍历image数组，将压缩之后的灰阶值保存在一个新数组cpimg中

```
// 计算整数c的压缩值
int compress(int c){
    int min=255,min_i=0;
    for(int h=0;h<16;h++){
        int k=abs(c-color[h]);
        if(k<min){
            min=k;
            min_i=h;}
    }
    return min_i;
}
```

```
int cpimg[25][25]; // 用于保存压缩图
int main(){
    .....
    int m=strlen(line)/2;
    for(int i=0;i<n;i++){
        for(int j=0;j<m;j++){
            cpimg[i][j]=compress(image[i][j]);
        }
    }
    return 0;
}
```



真题解析

思路分析

把图像字符串
转为**数值**



找出**最多的**
16种灰阶



根据规则
压缩图像



输出压缩后
图像的灰阶值



真题解析

实现步骤

- ⑤ 输出选取的16个灰阶值，即**color数组**中的元素
- ⑥ 输出压缩后图像的灰阶值，即**cpimg数组**中的元素
 - 输出的时候需要转换为十六进制字符的形式

63 → 3F

```
// 一位十六进制数字转换为字符
char itrans(int n) {
    .....
}
```

```
// 输出选取的16个灰阶值
for(int i=0;i<16;i++)
    cout<<color[i];
cout<<endl;
// 输出压缩后的图像
for(int i=0;i<n;i++){
    for(int j=0;j<m;j++){
        cout<<cpimg[i][j];
        cout<<endl;
    }
}
```

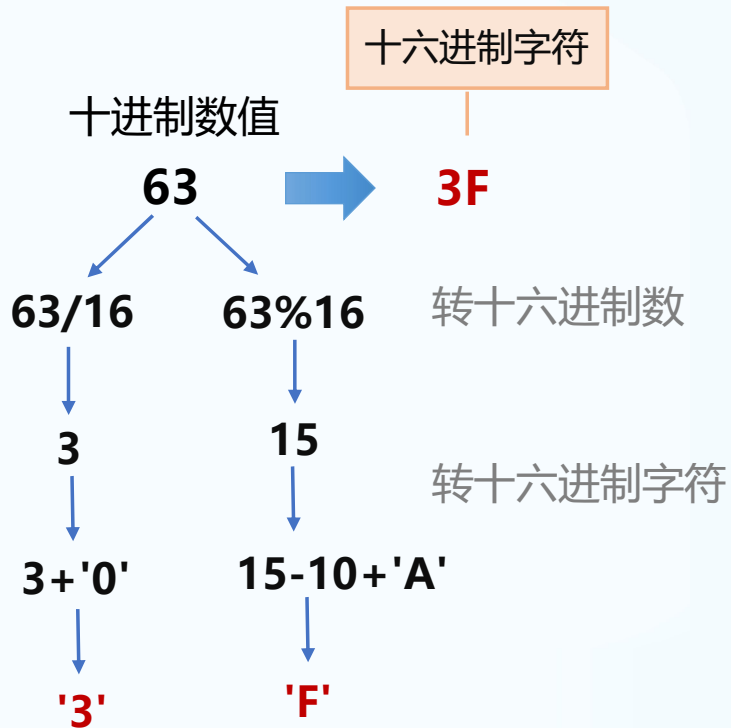




真题解析

itrans函数的实现

```
// 一位十六进制数字转换为字符  
char itrans(int n) {  
    .....  
}
```

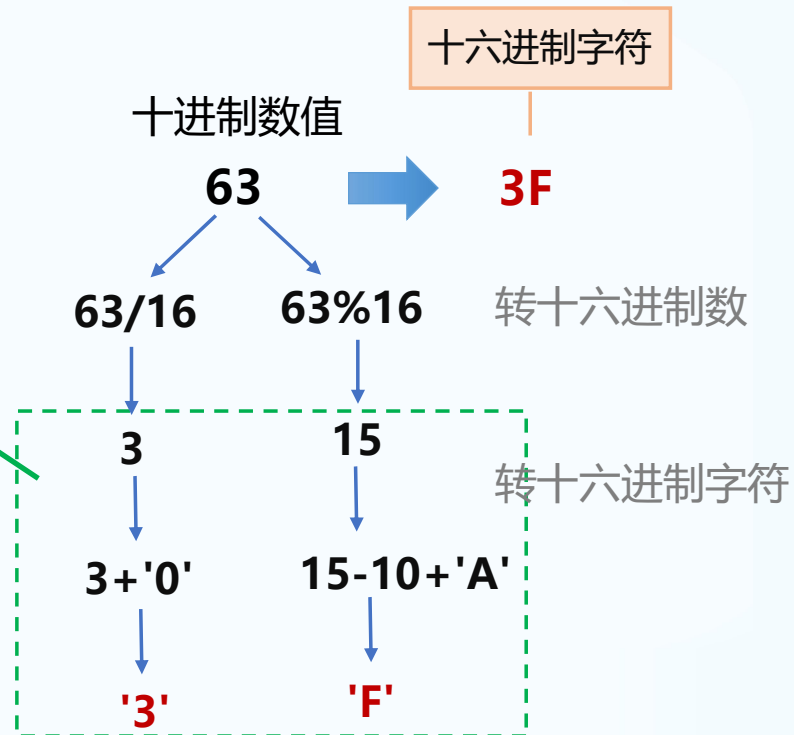




真题解析

itrans函数的实现

```
// 一位十六进制数字转换为字符  
char itrans(int n) {  
    if (n >= 10)  
        ...return (char)(n-10+'A');  
    return (char)(n+'0');  
}
```





真题解析

实现步骤

- ⑤ 输出选取的16个灰阶值，即**color数组**中的元素
- ⑥ 输出压缩后图像的灰阶值，即**cpimg数组**中的元素
 - 输出的时候需要转换为十六进制字符的形式

```
// 一位十六进制数字转换为字符
char itrans(int n) {
    if (n >= 10)
        return (char)(n - 10 + 'A');
    return (char)(n + '0');
}
```

```
// 输出选取的16个灰阶值
for (int i = 0; i < 16; i++)
    cout << itrans(color[i] / 16) << itrans(color[i] % 16);
cout << endl;
// 输出压缩后的图像
for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++)
        cout << itrans(cpimg[i][j]);
    cout << endl;
}
```



真题解析

【完整代码】

```
#include <bits/stdc++.h>
using namespace std;
// 一位十六进制字符转换为数字
int trans(char a) {
    if (a <= '9')
        return (a - '0');
    return (a - 'A' + 10);
}
// 一位十六进制数字转换为字符
char itrans(int n) {
    if (n >= 10)
        return (char)(n - 10 + 'A');
    return (char)(n + '0');
}
```

```
int image[25][25];
int cpimg[25][25];
int his[256];
int color[16];
// 计算整数c的压缩值
int compress(int c){
    int min=255,min_i=0;
    for(int h=0;h<16;h++){
        int k=abs(c-color[h]);
        if(k<min){
            min=k;
            min_i=h; //下标为转换值
        }
    }
    return min_i;
}
```



真题解析

【完整代码】

```
int main(){
    int n;
    cin>>n;
    char line[50]={0};
    for(int i=0;i<n;i++){
        cin>>line;
        for(int j=0;j<strlen(line);j+=2){
            int c = trans(line[j])*16+trans(line[j+1]);
            image[i][j/2]=c;
            his[c]++; // 统计灰阶值出现的次数
        }
    }
}
```



真题解析

【完整代码】

// 在his数组中找出最多的前16种灰阶值保存在数组color中

```
for(int i=0;i<16;i++){
    int max=0;
    for(int j=0;j<256;j++){
        if(his[max]<his[j]){
            max=j;
        }
    }
    color[i]=max;
    his[max]=-1;// 标记为-1, 该灰阶值已获取
}
// 对image数组中的元素进行压缩处理
int m=strlen(line)/2;
for(int i=0;i<n;i++){
    for(int j=0;j<m;j++){
        cpimg[i][j]=compress(image[i][j]);
    }
}
```




真题解析

【完整代码】

```
// 输出选取的16个灰阶
for (int i=0;i<16;i++)
    cout<<itrans(color[i]/16)<<itrans(color[i]%16);
cout<<endl;
// 输出压缩后的图像
for (int i=0;i<n;i++) {
    for (int j=0;j<m;j++)
        cout<<itrans(cpimg[i][j]);
    cout<<endl;
}
return 0;
}
```